2026-10-30 · 저장소의 HTML, main.js, Vite, Next.js 코드 기준

JS 호이스팅과 스크립트 로딩 비교 가이드

같은 메뉴 동작을 붙이는 방식이 파일마다 어떻게 다른지 비교한다. 호이스팅은 선언을 그 스코프 안에서 먼저 등록하는 규칙이고, 스크립트 로딩은 그 코드가 언제 다운로드되고 언제 실행되는가다. 둘은 따로 작동한다. 코드에 없는 동작은 적지 않았다.

---

1. 코드 지도

파일	스크립트	로딩	선언 방식
`index.html` (저장소 루트)	`<head>` 안 인라인 스크립트	고전 스크립트, 동기, 파서 차단	`function login()` 함수 선언
`lgcns-fe-html/index.html`	`</body>` 직전 `js/main.js`	고전 스크립트, 동기	`const` 와 화살표 함수
`lgcns-fe-html/about.html`	위와 같은 동기 태그	고전 스크립트, 동기	같은 `main.js`
`lgcns-fe-html/contact.html`	`async` 속성	고전 스크립트, async	같은 `main.js`
`lgcns-fe-html/login.html`	스크립트 없음	실행할 JS 없음	해당 없음
`lgcns-fe/index.html`	`type="module"` `main.tsx`	ES 모듈, 기본 defer	`import` 후 `createRoot`
`MobileNavigation.tsx`	소스에 script 태그 없음	Next 클라이언트 번들	`"use client"` , import, 함수 선언
`DesktopNavigation.tsx`	브라우저 스크립트 아님	서버 컴포넌트 렌더	import와 함수 선언

2. 호이스팅

호이스팅은 코드를 물리적으로 위로 옮기는 기능이 아니다. 자바스크립트는 스코프를 만들 때 선언을 먼저 등록한다. 등록된 뒤 바로 쓸 수 있는 선언과, 선언 줄을 실행하기 전에는 쓰면 안 되는 선언이 있다.

2.1 함수 선언은 스코프 전체에서 이름이 살아 있다

저장소 루트 `index.html` 은 `<head>` 에서 함수를 선언하고, 버튼은 그 아래 `<body>` 에 있다.

```
<script>
    function login() {
        const id = document.querySelector('.input01').value;
        const password = document.querySelector('.input01').value;
        console.log(id, password);
    }
</script>

<button type="button" class="mybtn" onclick="login()">LOGIN</button>
```

`function login()` 은 함수 선언이다. 이 스크립트 스코프에서는 함수 이름이 선언 줄보다 앞에서 호출돼도 연결된다. 고전 스크립트의 최상위 함수 선언은 전역 객체 속성 `window.login` 도 만든다. `onclick="login()"` 은 그 전역 이름을 찾아 호출한다.

함수 본문의 `document.querySelector`는 선언 시점이 아니라 버튼을 클릭한 뒤에 실행된다. 스크립트가 `<head>`에 있어 실행 당시에는 `.input01`이 아직 없어도, 클릭 시점에는 입력이 파싱된 뒤다. 호이스팅이 DOM을 미리 만들어 주는 것은 아니다. 함수 이름만 먼저 등록되고, DOM 조회는 호출 때까지 미뤄진다.

2.2 const는 등록만 되고, 선언 줄 전에는 임시 사각지대다

`lgcns-fe-html/js/main.js`에는 함수 선언이 없다.

```
const navToggle = document.querySelector(".nav-toggle");
const mobileMenu = document.querySelector(".mobile-menu");

if (navToggle && mobileMenu) {
  navToggle.addEventListener("click", () => {
    const isOpen = navToggle.getAttribute("aria-expanded") === "true";
    navToggle.setAttribute("aria-expanded", String(!isOpen));
    mobileMenu.hidden = isOpen;
  });
}
```

`const`도 스코프 시작 시 등록된다. 다만 선언 줄이 실행되기 전에는 초기화되지 않는다. 이 구간을 임시 사각지대(temporal dead zone)라고 한다. 그 구간에서 읽으면 `ReferenceError`다. 이 파일은 선언 다음에만 변수를 읽으므로 사각지대에 들어가지 않는다.

`const`는 `window` 속성으로 만들어지지 않는다. 같은 메뉴 함수를 `const login = () => {}`로 바꾸고 루트 페이지처럼 `onclick="login()"`에 기대면, `window.login`으로는 찾을 수 없다.

클릭 콜백은 화살표 함수다. 화살표 함수는 선언이 아니라 식이라 호이스팅되지 않는다. `addEventListener` 줄이 실행될 때 함수가 만들어지고, 그때의 `navToggle`과 `mobileMenu`를 닫아 둔다.

`if (navToggle && mobileMenu)`는 요소가 없을 때 오류 대신 리스너 등록을 건너뛴다. 스크립트가 요소보다 먼저 실행되면 메뉴는 조용히 동작하지 않는다.

2.3 모듈의 import는 파일 평가보다 먼저 연결된다

Vite 진입점 `lgcns-fe/src/main.tsx`는 import 다음에 `createRoot`를 호출한다. `import`는 모듈 스코프 맨 위에 있는 것으로 처리된다. 모듈은 본문을 실행하기 전에 의존 그래프를 연결한다. `createRoot` 줄은 그 연결이 끝난 뒤에 실행된다. 이 진입점에는 순환 참조가 없다.

```
"use client";
import Link from "next/link"
import { useState } from "react";

export default function MobileNavigation(){
    const [isOpen, setIsOpen] = useState(false);
```

`"use client"`는 자바스크립트 문법이 아니다. 파일 맨 위 지시어이고, Next 번들러가 이 파일을 브라우저 번들에 넣으라는 표시다. 호이스팅 대상이 아니다. 그 아래 `import`는 모듈 연결 단계에서 처리된다.

`export default function MobileNavigation`은 함수 선언이라 모듈 스코프 안에서 이름이 먼저 등록된다. `const [isOpen, setIsOpen]`은 함수가 호출되어 그 줄이 실행될 때 만들어진다. 렌더 전에 `isOpen`을 읽으면 임시 사각지대다.

`DesktopNavigation.tsx`에는 `"use client"`가 없다. `Link` import와 함수 선언만 있고, 이 함수 본문은 브라우저가 즉시 실행하는 스크립트가 아니다. Next가 서버에서 HTML로 그린다.

2.4 이 저장소에서 선언 방식만 모아 보면

선언	있는 곳	선언 줄 앞에서 이름을 쓸 수 있나	전역 window 속성
`function login()`	루트 `index.html`	같은 스크립트 안에서는 가능	생김
`const navToggle`	`lgcns-fe-html/js/main.js`	불가. 읽으면 ReferenceError	안 생김
화살표 함수	`main.js` 클릭 콜백, `CountBtn`	식이 실행될 때 생성	안 생김
`import`	`main.tsx`, Next 컴포넌트	모듈 본문 평가 전에 연결	모듈 스코프라 없음
`export default function`	MobileNavigation, DesktopNavigation	그 모듈 안에서는 가능	모듈 스코프라 없음

`var` 선언은 이 비교에 해당하는 소스에 없다.

3. 동기 실행과 즉시 실행

스크립트 태그의 기본값, `async`, `defer`, `type="module"`은 다운로드와 실행 시점을 바꾼다. 호이스팅 규칙 자체는 바꾸지 않는다.

3.1 동기 스크립트는 만나는 즉시 받고, 받는 동안 파서를 멈춘다

`lgcns-fe-html/index.html`과 `about.html`의 마지막은 속성 없는 고전 스크립트다.

```
<script src="js/main.js"></script>
```

파서가 이 태그를 만나면 `main.js`를 받고, 받는 동안 HTML 파싱을 멈춘 다음, 파일이 도착하면 바로 실행한다. 실행이 끝나야 뒤의 HTML을 계속 읽는다. 두 파일 모두 스크립트가 `</body>` 직전이라, 태그 앞의 `.nav-toggle`과 `.mobile-menu`는 이미 파싱된 상태다. `querySelector`가 요소를 찾고 클릭 리스너가 붙는다.

루트 `index.html`의 인라인 스크립트도 동기이다. 네트워크 요청은 없고, `<head>`를 읽는 도중에 바로 실행된다. 이때 함수 선언이 등록되고 함수 본문은 실행되지 않는다.

3.2 async는 파서를 멈추지 않고, 다운로드가 끝나면 바로 실행한다

`lgcns-fe-html/contact.html`만 속성이 다르다.

```
<script src="js/main.js" async></script>
```

`async` 스크립트는 다운로드를 파싱과 병렬로 진행하고, 파일이 준비되면 문서 파싱이 끝났는지를 기다리지 않고 실행한다. 실행 순서는 문서에 적힌 순서가 아니라 다운로드가 끝난 순서를 따른다. 이 페이지의 스크립트는 헤더 마크업 뒤에 있으므로, 태그를 만나는 시점에는 버튼이 이미 트리에 있다. 지금 마크업에서는 `querySelector`가 요소를 찾을 가능성이 크다.

즉시 실행이 문제가 되는 위치는 대상 요소보다 앞이다. 같은 `main.js`를 `<head>`로 올리거나, 버튼보다 앞에서 `async`로 실행되면 `navToggle`이 `null`이다. `if`가 거짓이라 예외는 나지 않고, 클릭 리스너만 빠진다. 메뉴가 열리지 않는다.

`defer`는 이 HTML 파일들에 없다. `defer`는 병렬로 받은 뒤 문서 파싱이 끝난 다음에, 문서 순서대로 실행한다.

3.3 ES 모듈은 defer가 기본이다

```
<div id="root"></div>
<script type="module" src="/src/main.tsx"></script>
```

`type="module"`은 `defer`를 적지 않아도 지연 실행이다. 다운로드는 병렬이고, 문서 파싱이 끝난 뒤 모듈 순서대로 실행된다. 그래서 `document.getElementById('root')`가 루트 요소를 찾은 다음 `createRoot`가 호출된다. 같은 파일을 속성 없는 고전 스크립트로 `<head>`에 두면, 실행 시점에 `#root`가 아직 없다.

모듈은 `async`를 붙이기 전까지 문서 순서를 지킨다. 고전 스크립트의 `async`와 이름이 같아도, 기본 동작은 `defer`에 가깝다.

3.4 Next는 소스에 스크립트 태그를 두지 않는다

`MobileNavigation`의 열림 상태는 `useState(false)`다. 첫 렌더에서 `isOpen`이 거짓이면 메뉴 `<ul>`은 만들어지지 않는다. 서버가 먼저 버튼 HTML을 보내고, 브라우저 번들이 하이드레이션된 뒤에 `onClick`이 붙는다. 클릭하면 `setIsOpen`이 상태를 바꾸고 React가 메뉴를 그린다.

정적 HTML은 반대로, 메뉴 DOM을 처음부터 `hidden`으로 두고 `main.js`가 `mobileMenu.hidden`만 바꾼다.

Next 서버 컴포넌트인 `Header`, `DesktopNavigation`, `app/(main)/layout.tsx`는 브라우저가 그 함수를 즉시 실행하지 않는다. 서버 렌더 결과만 내려온다. 클라이언트 경계는 `"use client"`가 있는 `MobileNavigation`이다.

3.5 로딩 방식 한눈에 보기

방식	예	다운로드	실행 시점	DOM
인라인 동기	루트 index.html의 head	없음	파서가 그 줄을 읽는 즉시	본문보다 앞. DOM 조회는 클릭 때
외부 동기	index.html, about.html	태그에서 즉시, 파서 정지	파일을 받자마자	태그 앞 요소는 이미 파싱됨
async	contact.html	병렬	받는 즉시, 파싱 완료를 기다리지 않음	요소보다 앞이면 리스너가 빠짐
type="module"	lgcns-fe/index.html	병렬	문서 파싱 후, 모듈 순서대로	#root 다음 createRoot
클라이언트 번들	MobileNavigation.tsx	Next가 주입	서버 HTML 이후 하이드레이션	메뉴 노드는 클릭 후 생성
스크립트 없음	login.html	없음	없음	JS로 바꿀 동작 없음

CSS `<link rel="stylesheet">`는 스크립트와 별도다. HTML 페이지의 스타일시트는 렌더를 막는다. 폰트 링크의 `preconnect`는 연결만 미리 연다.

4. 같은 메뉴가 갈라지는 지점

1. 정적 페이지는 `main.js`가 `querySelector`로 버튼을 찾고 `addEventListener`를 붙인다. 요소가 없으면 아무 일도 하지 않는다.
2. Next는 컴포넌트 함수가 렌더될 때 `useState`로 `isOpen`을 만든다. 리스너는 JSX의 `onClick`이다.
3. 정적 메뉴의 Home만 모바일 패널에 있다. 데스크톱 Home, About, Contact는 HTML 마크업에 처음부터 있다.
4. `login.html`은 인증 레이아웃만 있고 `main.js`를 불러오지 않는다.

5. 소스만 보고 확인하는 목록

- 루트 `index.html`에서 `function login`을 `const login = function () {}`로 바꾸면, 클릭 시 전역 속성 조회가 실패할 수 있다.
- `main.js` 1행 위에서 `navToggle`을 읽으면 `ReferenceError`다.
- `index.html`과 `about.html`은 동기라 파서가 `main.js` 동안 멈춘다. `contact.html`의 `async`는 파서를 멈추지 않는다.
- `contact.html`의 스크립트 태그를 버튼보다 앞으로 옮기면, 조건문이 거짓이 되어 메뉴가 열리지 않을 수 있다.
- `type="module"`을 빼면 지연 실행 보장이 사라진다. `#root`보다 앞에서 실행되면 `getElementById`가 `null`이다.
- `MobileNavigation.tsx`에서 `"use client"`를 빼면 그 파일의 `useState`는 서버 컴포넌트 규칙에 맞지 않는다. 지시어는 호이스팅이 아니다.

6. 공식 문서

언어와 선언

- MDN Hoisting
- MDN function 선언
- MDN let과 임시 사각지대
- MDN const
- ECMA-262 Let and Const
- ECMA-262 Function Definitions
- MDN JavaScript modules
- ECMA-262 Imports

스크립트 로딩

- MDN script 요소
- HTML 명세 script 요소
- HTML 명세 async
- HTML 명세 defer
- MDN DOMContentLoaded

이 저장소의 프레임워크

- Vite index.html과 모듈 스크립트
- React useState
- Next.js use client
- Next.js Script 컴포넌트
- React 서버 컴포넌트